

Oriented Bounding Box Object Detection

Oriented Bounding Box Object Detection

- 1. Model Introduction
- 2. Oriented Bounding Box Object Detection: Image
Effect Preview
- 3. Oriented Bounding Box Object Detection: Video
Effect Preview
- 4. Oriented Bounding Box Object Detection: Real-time Detection
 - 4.1. USB Camera
Effect Preview
 - 4.2. CSI Camera
Effect Preview
- Reference Materials

Use Python to demonstrate **Ultralytics**: Oriented Bounding Boxes Object Detection effects on images, videos, and real-time detection.

1. Model Introduction

Oriented bounding box object detection, also called oriented object detection, goes a step further than standard object detection by introducing an additional angle to more accurately locate objects in images.

The output of an oriented object detector is a set of rotated bounding boxes that precisely surround objects in the image, along with class labels and confidence scores for each box. Oriented bounding boxes are particularly useful when objects appear at different angles, such as in aerial images, where traditional axis-aligned bounding boxes might include unnecessary background.

Simply put, oriented object detection can use **tilted boxes** to precisely surround **tilted objects**, thereby reducing unnecessary background areas and improving detection accuracy.

2. Oriented Bounding Box Object Detection: Image

Use yolo26n-obb_ncnn_model to predict images in the ultralytics26 folder (not built-in ultralytics images).

Enter the code folder:

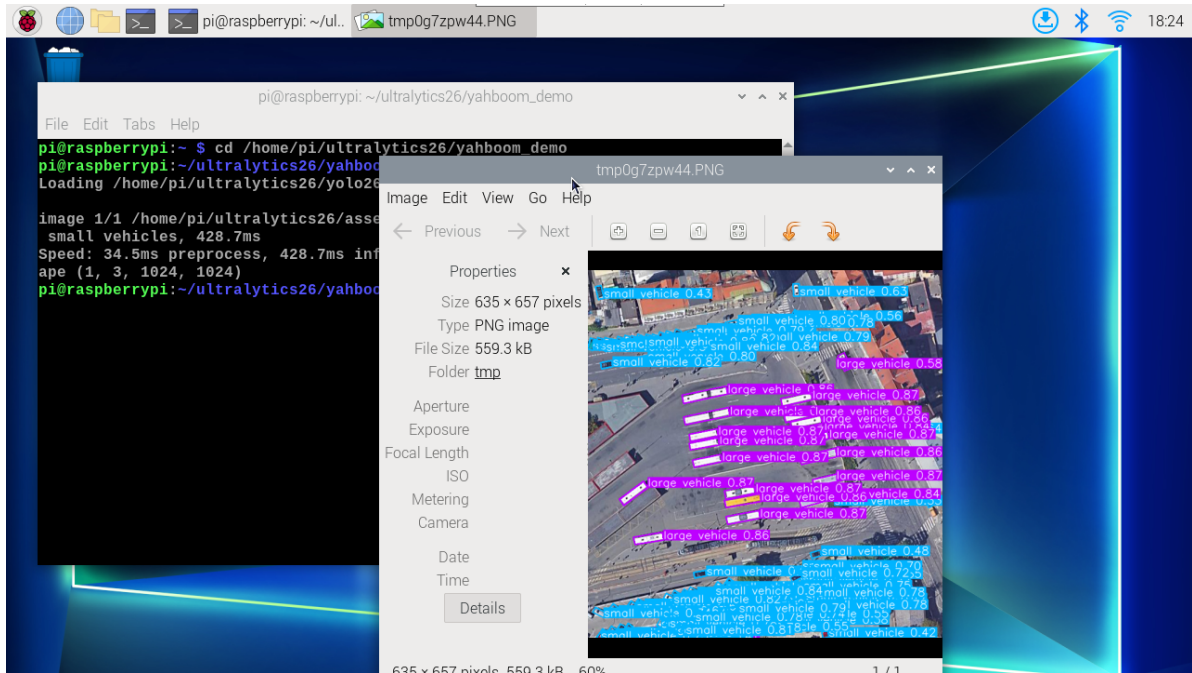
```
cd /home/jetson/ultraLytics/yahboom_demo
```

Run the code:

```
python3 05.obb_image.py
```

Effect Preview

yolo recognition output image location: /home/jetson/ultralytics/output/



Example code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/home/jetson/ultralytics/yolo26n-obb.engine")

# Run batched inference on a list of images
results = model("/home/jetson/ultralytics/assets/bus.jpg") # return a list of Results objects

# Process results list
for result in results:
    # boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    obb = result.obb # Oriented boxes object for OBB outputs

    # Print OBB results
    if obb is not None:
        print("OBB boxes:", obb.xyxy)
        print("OBB angles:", obb.angle)
        print("OBB classes:", obb.cls)
        print("OBB confidences:", obb.conf)

    result.show() # display to screen
    result.save(filename="/home/jetson/ultralytics/output/bus_obb.jpg") # save to disk
```

3. Oriented Bounding Box Object Detection: Video

Enter the code folder:

Run the code:

Effect Preview

0: 1024x1024 1 large vehicle, 4 small vehicles, 29.9ms
Speed: 11.8ms preprocess, 29.9ms inference, 1.7ms postprocess per image at shape (1, 3, 1024, 1024)

0: 1024x1024 1 large vehicle, 4 small vehicles, 21.3ms
Speed: 11.6ms preprocess, 21.3ms inference, 1.6ms postprocess per image at shape (1, 3, 1024, 1024)

0: 1024x1024 1 large vehicle, 8 small vehicles, 29.9ms
Speed: 11.9ms preprocess, 29.9ms inference, 1.7ms postprocess per image at shape (1, 3, 1024, 1024)

0: 1024x1024 1 large ve
Speed: 11.9ms preproces

0: 1024x1024 1 large ve
Speed: 12.3ms preproces

0: 1024x1024 1 large ve
Speed: 12.3ms preproces

0: 1024x1024 1 large ve
Speed: 11.6ms preproces

0: 1024x1024 1 large ve
Speed: 12.1ms preproces

0: 1024x1024 1 large ve
Speed: 12.1ms preproces

0: 1024x1024 1 large ve
Speed: 12.4ms preproces

0: 1024x1024 3 large ve
Speed: 11.6ms preproces

0: 1024x1024 1 large vehicle, 4 small vehicles, 32.3ms
Speed: 11.8ms preprocess, 32.3ms inference, 1.3ms postprocess per image at shape (1, 3, 1024, 1024)

0: 1024x1024 1 large vehicle, 5 small vehicles, 36.7ms
Speed: 11.7ms preprocess, 36.7ms inference, 2.0ms postprocess per image at shape (1, 3, 1024, 1024)

0: 1024x1024 1 large vehicle, 6 small vehicles, 34.7ms
Speed: 12.2ms preprocess, 34.7ms inference, 2.0ms postprocess per image at shape (1, 3, 1024, 1024)

The screenshot displays a video analysis application. On the left, a terminal window titled "YOLO OBB@yahboom" shows a series of command-line outputs detailing the model's performance across various test cases, such as detecting 1 large and 4 small vehicles in 29.9ms. The main area features a video frame of a road intersection with several cars. Bounding boxes are drawn around these cars, each containing a label like "small vehicle 0.39" or "large vehicle 0.66". To the right of the video frame, there are vertical bars representing the detection results for each frame.

```
import cv2
from ultralytics import YOLO

# Load the YOLO OBB model
model = YOLO("/home/jetson/ultralytics/yolo26n-obb.engine")

# Open the video file
video_path = "/home/jetson/ultralytics/ultralytics/videos/people_animals.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/home/jetson/ultralytics/output/05.obb_video_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
```

```

out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO OBB inference on the frame
        results = model(frame)

        # Visualize the OBB results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO OBB", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4. Oriented Bounding Box Object Detection: Real-time Detection

4.1. USB Camera

Use yolo26n-obbb_ncnn_model to predict USB camera footage.

Enter the code folder:

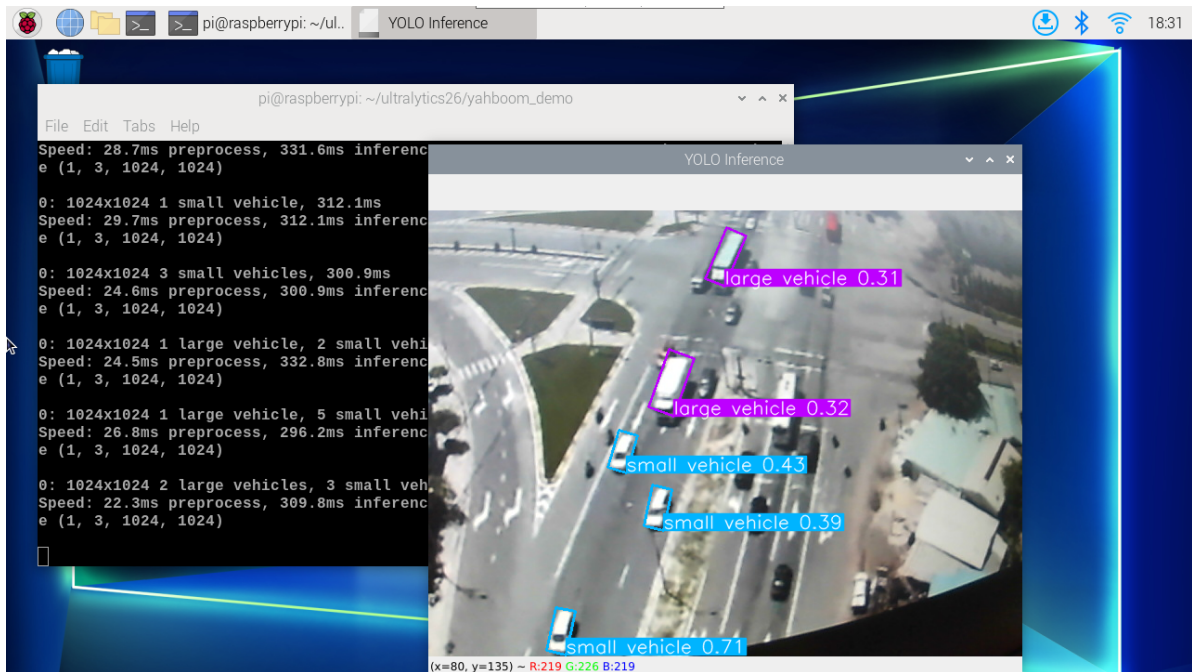
```
cd /home/jetson/ultralytics/yahboom_demo
```

Run the code: Click on the preview screen, press q key to terminate the program!

```
python3 05.obb_camera_usb.py
```

Effect Preview

yolo recognition output video location: /home/jetson/ultralytics/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO OBB model
model = YOLO("/home/jetson/ultralytics/yolo26n-obb.engine")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/jetson/ultralytics/output/05.obb_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO OBB inference on the frame
        results = model(frame)

        # Visualize the OBB results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
```

```

out.write(annotated_frame)

# Display the annotated frame
cv2.imshow("YOLO OBB", cv2.resize(annotated_frame, (640, 480)))

# Break the loop if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    # Break the loop if the end of the video is reached
    break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4.2. CSI Camera

Use yolo26n-obb_ncnn_model to predict CSI camera footage.

Enter the code folder:

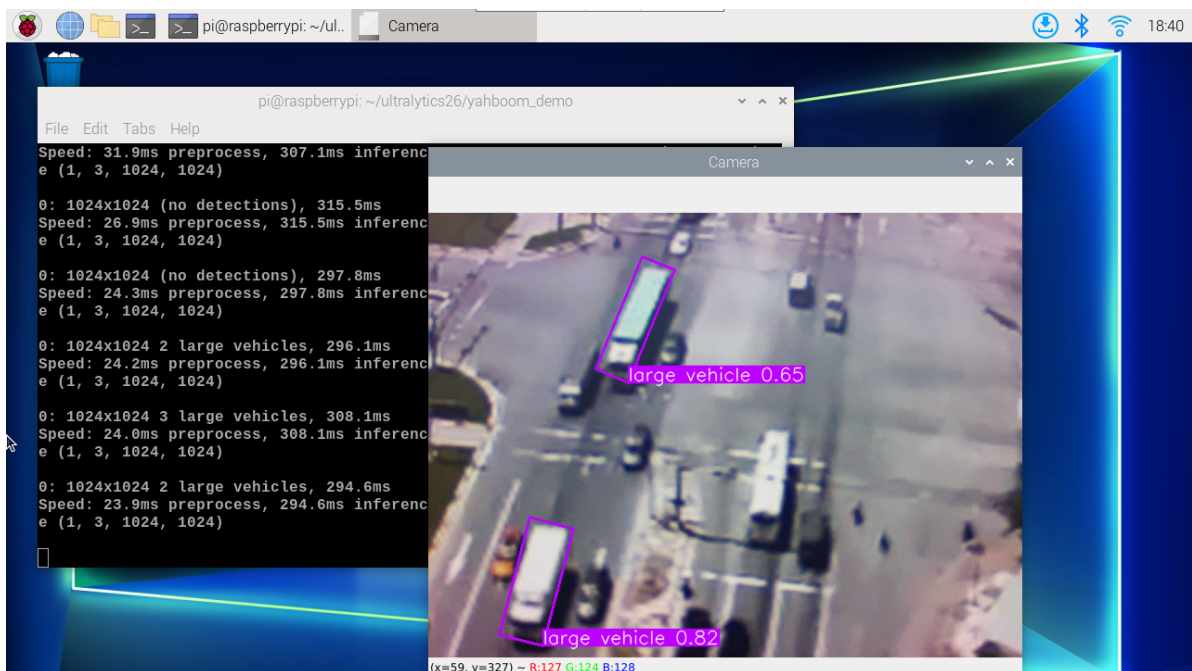
```
cd /home/jetson/ultralytics/yahboom_demo
```

Run the code: Click on the preview screen, press q key to terminate the program!

```
python3 05.obb_camera_csi.py
```

Effect Preview

yolo recognition output video location: /home/jetson/ultralytics/output/



Example code:

```

import cv2
from ultralytics import YOLO

```



```

from jetcam.csi_camera import CSICamera

# Load the YOLO OBB model
model = YOLO("/home/jetson/ultralytics/yolo26n-obb.engine")

# Open the camera (CSI Camera)
cap = CSICamera(capture_device=0, width=640, height=480)

# Get the video frame size and frame rate
frame_width = 640
frame_height = 480
fps = 30

# Define the codec and create a Videowriter object to output the processed video
output_path = "/home/jetson/ultralytics/output/05.obb_camera_csi.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while True:
    # Read a frame from the camera
    frame = cap.read()

    if frame is not None:
        # Run YOLO OBB inference on the frame
        results = model(frame)

        # Visualize the OBB results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO OBB", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if no frame is received (camera error or end of stream)
        print("No frame received, breaking the loop.")
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

Reference Materials

[Oriented Bounding Box Object Detection - Ultralytics YOLO Documentation](#)

